Word Ladder (hard)

Problem Statement

A `transformation sequence` is a sequence that starts from the word `beginWord`, ends with `endWord`, and contains a word from the given dictionary `wordList` in the middle i.e. a sequence of words `beginWord -> s1 -> s2 -> ... -> sk` such that:

- Each adjacent pair of words differs by a single letter.
- Each si for `1 <= i <= k` is in wordList. sk == endWord
- The `beginWord` does not need to be in wordList.

Given two strings, `beginWord` and `endWord`, and a `wordList`, return the number of words in the shortest transformation sequence from beginWord to endWord, or -1 if no such sequence exists.

Examples

- **Example 1**:
  - **Input**: startWord = "cat", endWord = "dog", wordList = ["cot", "dot", "dog"]
  - **Expected Output**: 4
  - **Justification**: The shortest transformation sequenc is "cat" -> "cot" -> "dot" -> "dog", which has 4 words.
- **Example 2**:
  - **Input**: startWord = "bit", endWord = "pog", wordList = ["but", "put", "pig", "pog", "big"]
  - **Expected Output**: 4
  - **Justification**: The shortest transformation sequenc is "bit" -> "big" -> "pig" -> "pog", which has 4 words.
- **Example 3**:
  - **Input**: startWord = "sing", endWord = "ring", wordList = ["ring"]
  - **Expected Output**: 2
  - **Justification**: The shortest transformation sequenc is "sing" -> "ring", which has 2 words.

Try it yourself

Try solving this question here:

Java

```java
import java.util.*;

public class Solution {
    // Method to find the shortest transformation sequence length
    public int ladderLength(String beginWord, String endWord, List<String> wordList) {
        // ToDo: Write Your Code Here.
        return -1; // Transformation not possible
    }
}
```

Test Case | Run Code Result | Stdout

```
"cat"
"dog"
["cot", "dot", "dog"]
```

Use Example Testcases | Run | Submit

← Previous
Solution: Count Unique Characters Of All Substrings Of A Given String

Next →
Solution: Word Ladder

Mark as Completed